

Rapport de projet

RplaceClicker

Auteurs : Lacaton Damien, Bouzom Lukas, Charrier Titouan

Professeure : Léa Brunschwig

Dépot de code : DWA2026_DamienLacaton_TitouanCharrier_LukasBouzom

Date : 18 mai 2026



Projet réalisé dans le cadre du second semestre du Master Technologie de l'Internet à l'Université de Pau et Pays de l'Adour.

Table des matières

1	Introduction	2
1.1	Préambule	2
1.2	Sujet	2
2	Choix et justification des technologies	3
2.1	Backend	3
2.1.1	général	3
2.1.2	Base de donnée	3
2.1.3	Serveur Java	3
2.1.4	Couche d'abstraction SQL	4
2.2	Frontend	4
2.2.1	Serveur de front	4
2.2.2	Framework	4
3	Utilisation du projet	4
3.1	Lancement	4
3.2	Tutoriel	5
3.2.1	Rplace	5
3.2.2	Clicker	9
4	Gestion des données	11
4.1	Schéma entité-association	11
4.2	Schéma relationnel	13
4.3	Transcription en objet avec JPA	15
5	Déploiement	17
6	Répartition du travail	17
6.1	Damien (Chef de projet + Lead dev)	17
6.2	Lukas (Dev + UI Designer)	17
6.3	Titouan (DevOps)	18

1 Introduction

1.1 Préambule

1.2 Sujet

Règles du jeu Principe général

Chaque utilisateur a en face de lui une grille de 50*50 pixels qui est accessible par tous et dont l'évolution est observable en temps réel. Ainsi, chaque pixel posé par un utilisateur authentifié est immédiatement visible par tous les autres utilisateurs. Pour poser un pixel, un utilisateur doit avoir créé un compte et s'être connecté autrement il ne sera que spectateur de la grille. Poser un pixel coûte des « crédits » et ces derniers se régénèrent lentement avec le temps ou bien au travers d'actions du joueur. Le prix d'un pixel dépend de son emplacement : si le pixel a déjà été coloré, son prix augmente donc sa popularité fait augmenter son prix.

L'accès à la grille n'est pas limité à un nombre d'utilisateur et il n'y a pas non plus de système de tour par tour, ainsi la modification de la grille se fait en concurrence. Déroulement du jeu

Un utilisateur non connecté verra la grille être modifiée en temps réel et pourra consulter qui a posé chaque pixel ainsi que le classement des utilisateurs en temps réel mais ne pourra pas interagir.

Un utilisateur authentifié pourra poser des pixels, miner des crédits manuellement en cliquant sur un bouton ou bien optimiser la récolte de ces crédits en achetant des bonus. Il est aussi possible d'attendre passivement que sa cagnotte de crédit augmente. Fonctionnalités et Interface Gestion des comptes joueur

Chaque joueur peut se créer un compte en précisant ses informations personnelles : pseudo, mot de passe, âge et pays. A l'exception du pseudo, les autres champs peuvent être modifiés. Interface de jeu

Chaque joueur peut se connecter avec son pseudo et son mot de passe.

Tous les utilisateurs peuvent voir la grille en temps réel et doivent pouvoir connaître l'auteur d'un pixel (en survolant le pixel par exemple) ainsi que le classement.

Seuls les utilisateurs authentifiés peuvent voir le prix de chaque pixel, leurs nombres de crédits ainsi que les fonctionnalités débloquées ou disponibles du cookie clicker. Le prix d'un pixel vierge coûte 10 crédits. Son prix augmente de 5 crédits dès que l'on souhaite le recouvrir. Ainsi, un pixel qui a déjà été coloré 3 fois coûte 25 crédits. Il est possible de changer la couleur d'un pixel au moment de son achat. Fonctionnalité Cookie Clicker

Par défaut, la cagnotte d'un joueur augmente d'un crédit toutes les 10 secondes ou bien d'un crédit à chaque clique d'un bouton particulier. Avec les crédits il est possible d'acheter des bonus de deux sortes :

des générateurs de crédits (cf. Cookie Clicker et ses curseurs, ses grands-mères, ses fermes et cie) des achats de couleurs supplémentaires pour les pixels ou des curseurs plus large pour acheter plus de pixels d'un coup (4 pixels colorés en un seul clic)

Statistiques

Un ensemble d'informations devront être disponibles :

La liste des joueurs enregistrés, Des informations sur chaque joueur : en plus des informations personnelles, on précisera le nombre de pixels placés depuis le début, le nombre de pixels encore en place, le record d'âge pour un pixel (il peut ne plus exister), le pixel le plus ancien encore en place du joueur. Le résultat du r/Place en cours : liste ordonnée des joueurs avec le pourcentage de couverture de la grille, liste ordonnée des joueurs dans l'ordre du plus vieux pixel par joueur.

Règles du jeu

Les règles du jeu doivent être disponibles à chaque instant quelque part sur le site afin que tout nouveau joueur puisse les consulter pour participer au r/Place. Contraintes techniques

Parties données

Toutes les données sur les joueurs et les parties seront stockées dans une base de données relationnelle dont l'accès se fera via le framework JPA et/ou JDBC. La couche d'accès aux données doit être totalement découplée des couches métier et de génération/gestion des pages Web. Parties web

La partie Web prendra en charge la génération des interfaces du jeu. Les langages utilisés pour les pages Web seront HTML5 et CSS3.

Pour les interactions, les technologies utilisées seront celles vues en cours : JSP, Javascript, ... L'utilisation de frameworks n'est pas obligatoire mais pas interdite non plus.

2 Choix et justification des technologies

2.1 Backend

2.1.1 général

Pour le backend de ce projet nous avons choisi d'utiliser un docker compose, comprenant un container pour la base de donnée, un container pour le serveur java. Nous avons choisi cela pour des raisons de portabilité, de facilité de déploiement mais également de sécurité. Nous y reviendrons dans la partie déploiement mais comme le projet est déployé, en cas de faille de sécurité de notre part sur le serveur java ou la base de donnée, seul le container et son volume seront compromis.

2.1.2 Base de donnée

Pour la base de donnée nous sommes restés sur une simple mysql c'est un bon compromis entre le minimalisme de SQLite et le côté complexe et *production ready* de PostgreSQL. Nous avons choisi une base de donnée relationnelle pour nous conformer au sujet.

2.1.3 Serveur Java

Pour notre serveur nous avons choisi SpringBoot, c'est un projet moderne qui inclut les bases de la sécurité, cela s'inscrit dans le choix d'utiliser des technologies solides au lieu de réinventer la roue et de produire des failles de sécurité évitables. Spring Boot est assez lourd, et consomme beaucoup de mémoire comparé à ses concurrents tels que Quarkus ou Micronaut mais sa communauté est beaucoup plus active, il possède la meilleure documentation et des starters prêts à l'emploi, c'est pourquoi nous l'avons choisi.

2.1.4 Couche d'abstraction SQL

Nous avons choisi d'utiliser JPA afin de nous conformer au sujet et d'avoir un accès aux données performant et facile d'utilisation.

2.2 Frontend

2.2.1 Serveur de front

Pour le développement nous avons choisi d'utiliser **Vite** un serveur de développement frontend qui permet une mise à jour en directe des modifications ainsi qu'une vue de debug rapide. Comme vite est un SPA (Single Page Application), cela permet de transférer le calcul du jeu au client et donc soulager notre infrastructure. En terme de SPA nous avons le choix entre **Vite** et **Webpack**. Bien que **Webpack** soit plus complet et plus mature, il est également plus difficile à déployer et à apprendre. Nous ne sommes pas dans une production industrielle donc nous avons choisi la simplicité de **Vite**.

2.2.2 Framework

Nous avons choisi d'utiliser le framework **Vue** car c'est un framework moderne et rapide, qui est plus simple d'utilisation qu'**Angular** donc plus adapté pour notre cas. Cela nous permet d'avoir une architecture claire, maintenable et facilement upgradable.

3 Utilisation du projet

Si vous voulez utiliser le projet déployé par nos soins vous pouvez vous rendre sur

```
https://dwa.charrier.ovh
```

Si c'est le cas, omettez la partie **Lancement** et passez directement à la partie **Tutoriel**

3.1 Lancement

Pour lancer le projet, commencez par cloner le projet avec la commande

```
git clone https://git.univ-pau.fr/tcharrier003/  
DWA2026_DamienLacaton_TitouanCharrier_LukasBouzom.git
```

Se rendre ensuite à la racine du projet et procéder comme suit (copier-coller possible) :

```
# Pour le backend
cd backend

# Compiler le serveur java (java 17 requis)
cd rplaceserv
mvn clean package
cd ..

# Configurer le .env
cp .env.example .env

# Lancer le backend general
docker compose up -d

# Pour le frontend
cd ../frontend/rplace-client

# Configurer le .env
cp .env.example .env

# Installer les dependances npm
npm install

# Lancer le serveur
npm run dev
```

Attention, si après le copier-coller vous avez une erreur docker, c'est souvent que le conteneur à été lancé avant la fin de la compilation du serveur java, vous pouvez relancer le script entier ou simplement la partie docker.

Vous pouvez ensuite vous rendre sur la fenêtre affiché dans Vite, normalement :

```
http://localhost:5173/
```

Attendez 1 à 2 min le temps que le serveur et la BDD démarrent.

3.2 Tutoriel

3.2.1 Rplace

Une fois arrivé sur la page d'accueil, vous pouvez voir en haut à gauche un encart **Invité** en cliquant dessus vous pourrez vous connecter ou créer un compte

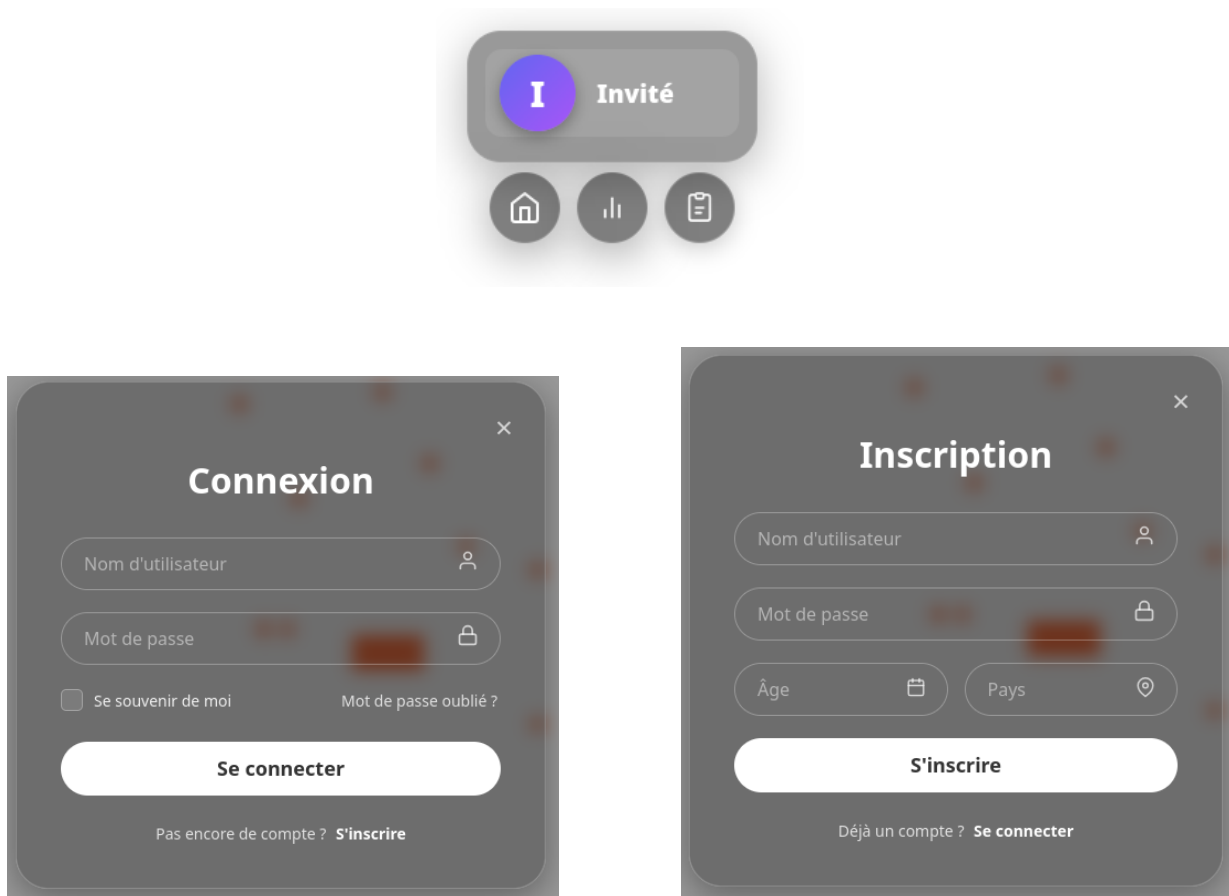


FIGURE 1 – Modules de connexion et inscription

Comme demandé, sans connexion les règles sont disponible à travers l'icône règles.

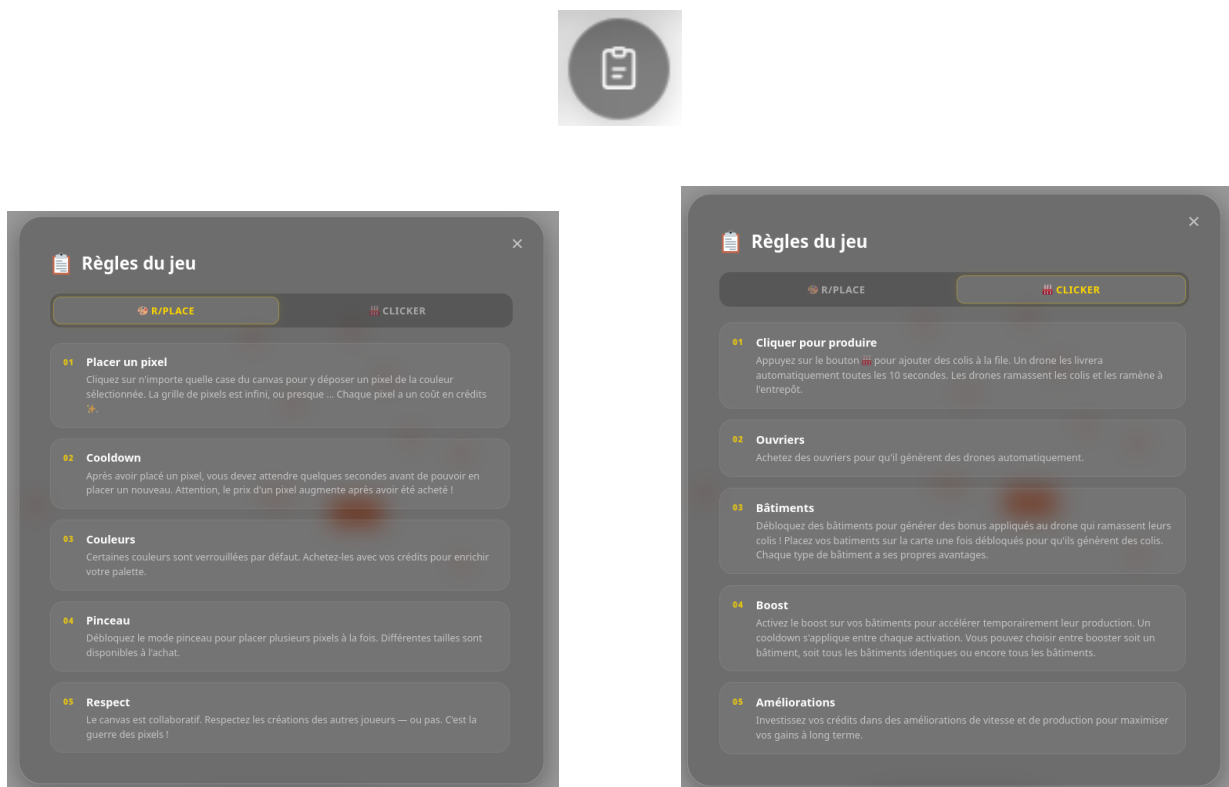


FIGURE 2 – Modules de connexion et inscription

Tout comme le classement

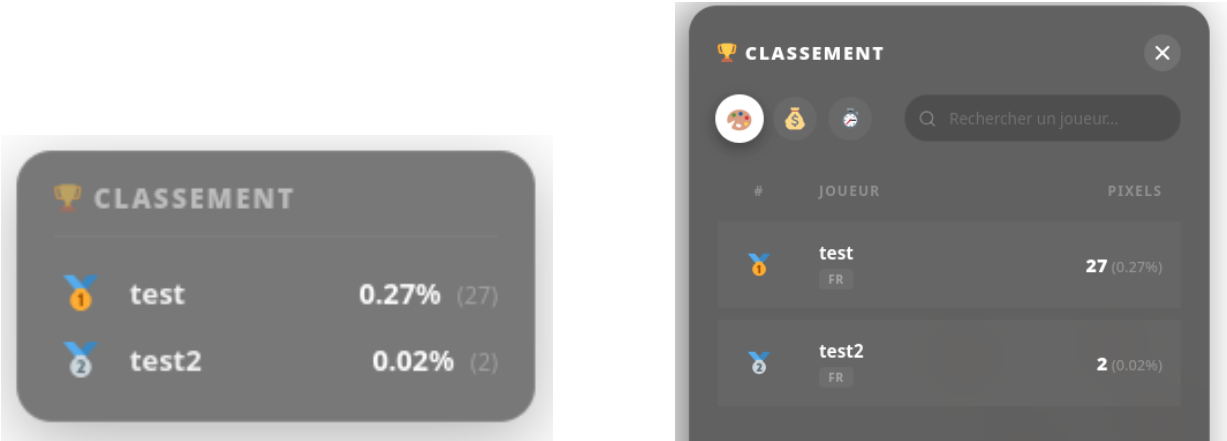


FIGURE 3 – Modules du classement

Dans la figure 3, on peut voir le classement où l'on peut choisir de trier les joueurs en fonction de leur nombre de pixel, argent détenu, record d'âge de pixel. Grâce aux boutons situés en haut à gauche. On peut également chercher à travers la liste avec la barre de recherche. Dans le classement miniaturisé on peut voir le pourcentage de la map détenue.

Si on clique sur un joueur dans le classement, on peut voir ses statistiques publiques :

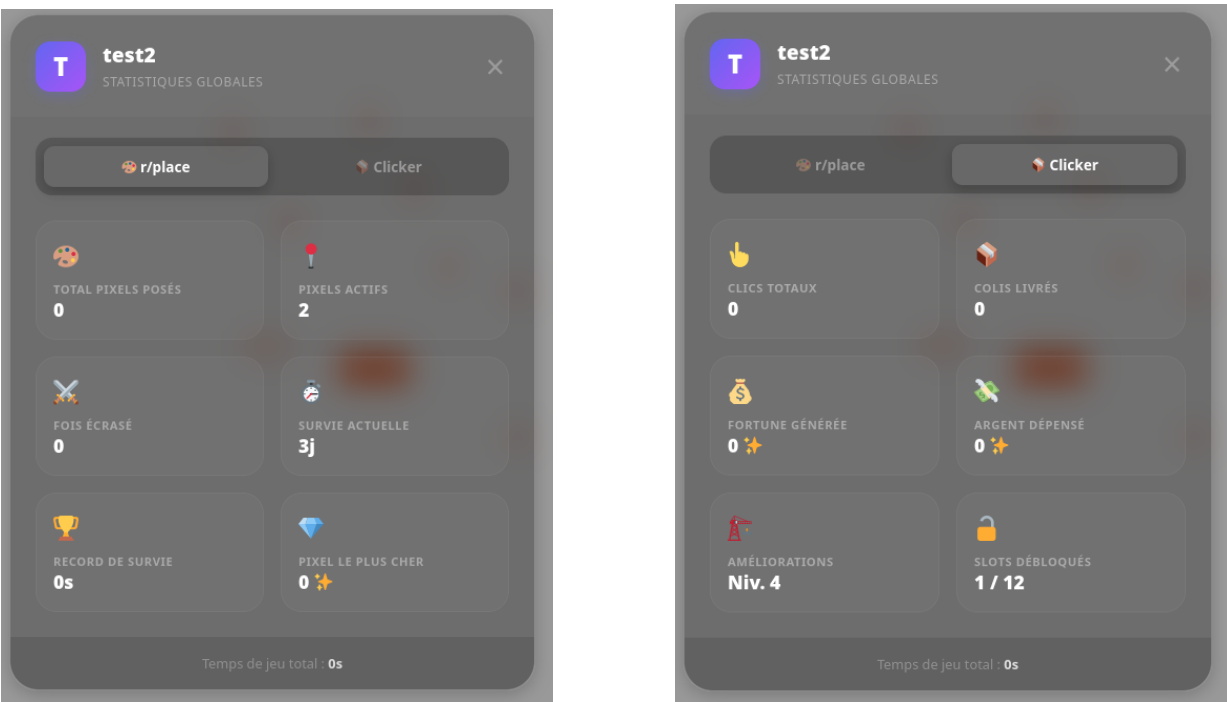
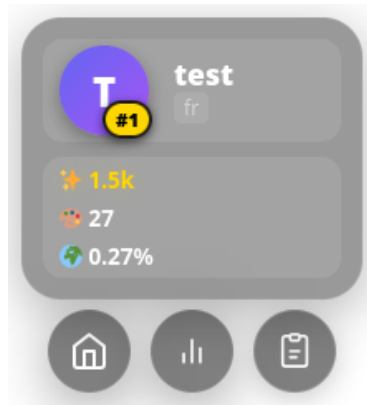
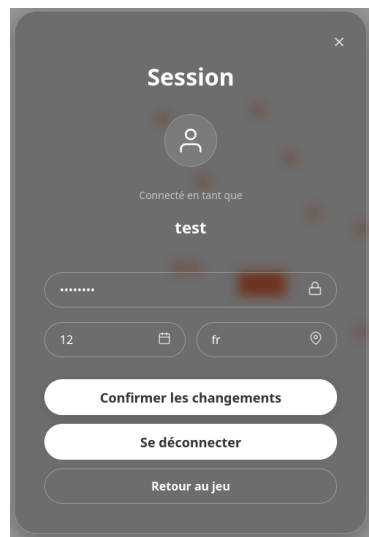


FIGURE 4 – Module de statistiques

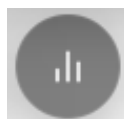
Note : Si des incohérences sont relevées dans les images de ce rapport, elles sont dues à une base de données de développement incorrecte, elles ne reflètent pas l'état final du projet. Une fois connecté, votre profil s'affiche sous cette forme :



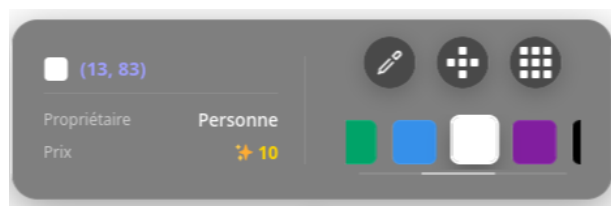
Avec de haut en bas, votre argent, les pixels détenus, le pourcentage de la map occupé. En cliquant dessus vous pouvez mettre à jour vos informations personnels



Vous pouvez aussi retrouver vos statistiques grâce au bouton :



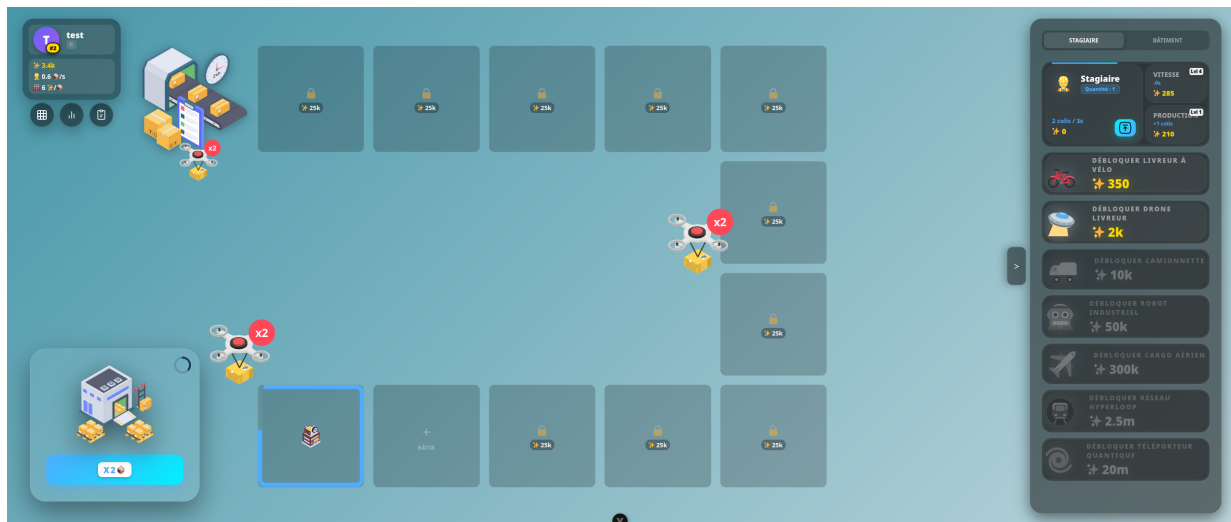
Vous pouvez maintenant poser des pixels sur la map grace à la palette de couleur :



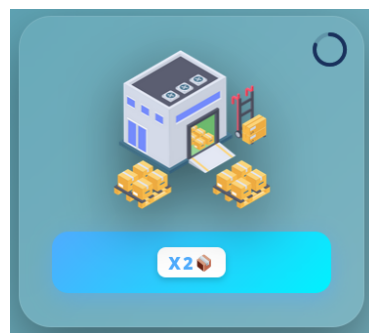
Cet encart vous permet de choisir non seulement la couleur de votre pixel mais aussi de sélectionner / acheter des *pinceaux* afin de poser plusieurs pixels à la fois sur la map.

3.2.2 Clicker

Quand vous cliquez sur l'icone *maison* sous le profil une fois connecté, vous parvenez à l'espace clicker :



Pour commencer à jouer, vous devez cliquer sur le bouton usine afin de produire des colis :



Les colis seront livrés par drone et acheminé jusqu'à l'entrepôt en ramassant sur le passage des bonus produits par les bâtiments :

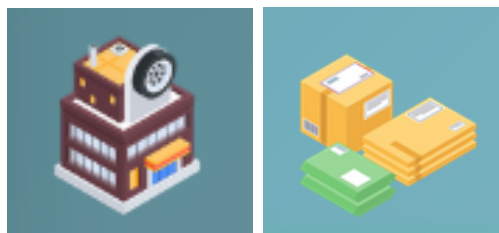


FIGURE 5 – bâtiments et bonus

Pour acheter des bâtiments ou des travailleurs (*stagiaires*) Vous devez vous rendre sur le panneau de droite :

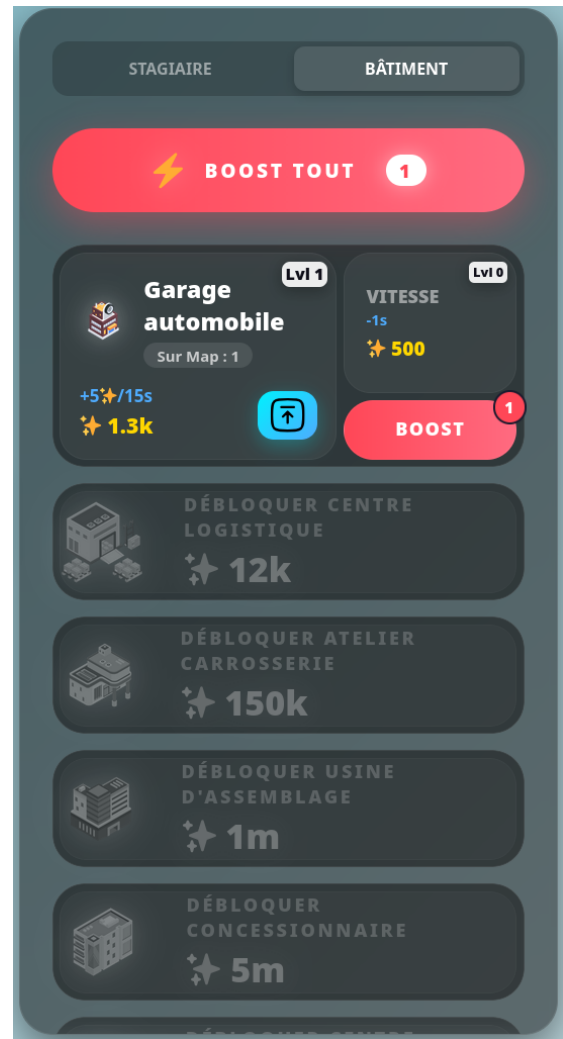
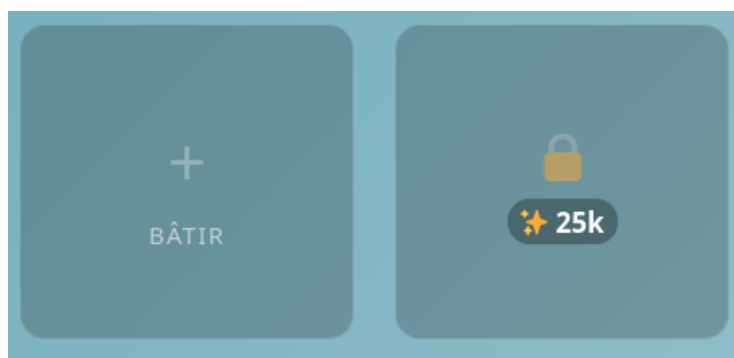


FIGURE 6 – Panneau d'achat Ouvriers et Bâtiment

Pour les ouvriers, nous pouvons débloquent de nouveaux types, augmenter leur vitesse ou leur production.

Pour les bâtiment nous pouvons débloquent de nouveaux types, augmenter leur vitesse ou activer un boost, qui va augmenter leur productivité durant un certain temps.

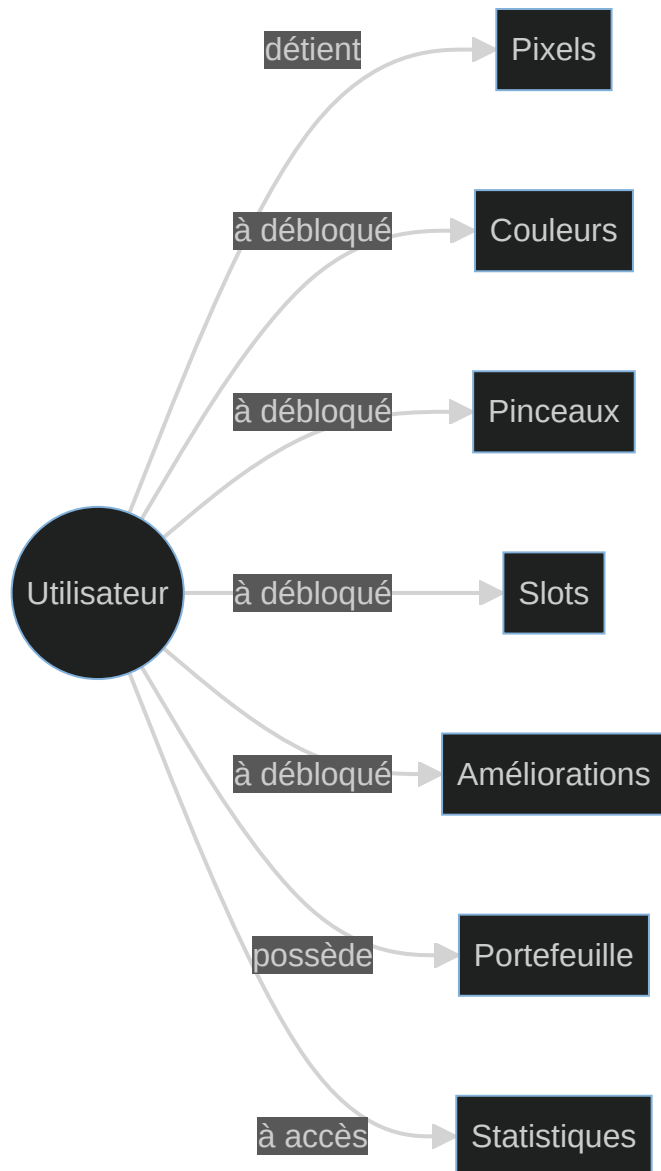
Une fois les bâtiments débloqués, on peut les installer sur des slots, les slots peuvent être acheté pour de l'argent.



4 Gestion des données

4.1 Schéma entité-association

Dans le schéma ci dessous nous pouvons voir les associations entre l'utilisateur et et les différentes parties de notre projet. Nous avons choisi une approche centré sur l'Utilisateur, le schéma est donc simple.



4.2 Schéma relationnel

Dans ce Schéma relationnel, nous avons fait le choix de centrer toutes les tables autour de l'utilisateur.

La table user stocke les informations relatives au compte de l'utilisateur. La table Pixels Stocke les informations relatives à chaque pixel de la grille, chaque pixel peut être lié à entre 0 et 1 utilisateur.

La table slots stocke la liste des emplacements de batiments qui ont été débloqués pour chaque utilisateur.

La table Wallets stocke l'argent et le records enregistré de durée des pixel de l'utilisateur.

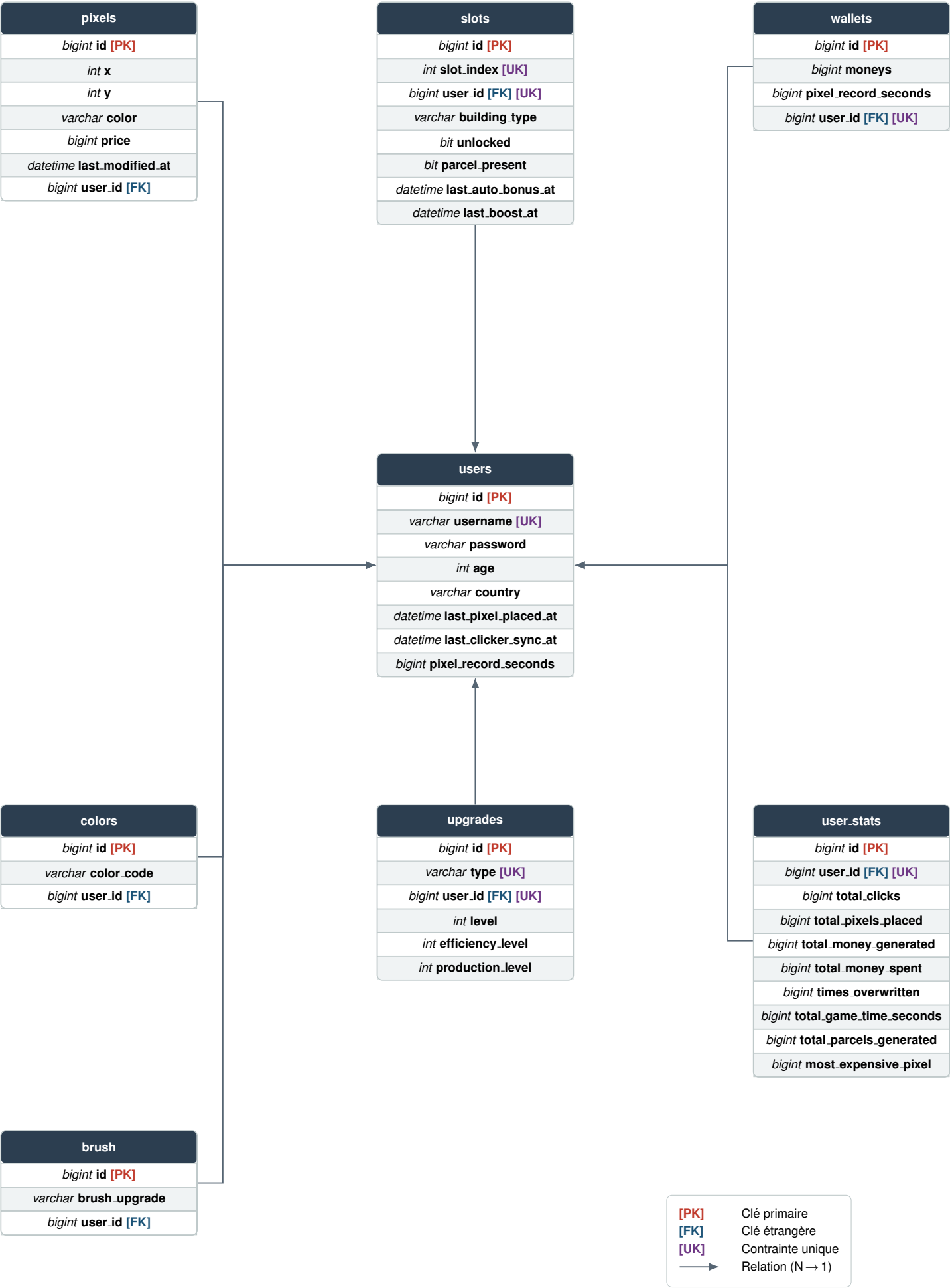
Dans la tables color nous stockons la liste des couleurs débloqués par l'utilisateur

Dans la tables upgrades nous stockons les amélioration des bâtiments / ouvriers débloqués par l'utilisateur.

Dans la table brush, nous stockons les pinceaux débloqués par l'utilisateur.

Dans la tables user_stats nous stockons les statistiques relatives a chaque utilisateur.

Diagramme Entité-Relation — RPlace Clicker



4.3 Transcription en objet avec JPA

Ci dessous, vous trouverez un exemple de transcriptions de notre Table User en JPA. Nous utilisons les notations vues en cours.

De plus,

`@NoArgsConstructor` permet d'éviter d'avoir à ajouter un constructeur vide.

`@AllArgsConstructor` permet de créer un objet directement en une seule ligne.

`@OneToOne` signifie que User et Wallet ont une relation 1 pour 1. Le `mappedBy = "user"` indique que c'est Wallet qui porte la clé étrangère.

`@ToString.Exclude` exclut ce champ du `toString()` généré par `Data`, pour éviter la boucle infinie User -> Wallet -> User -> ... lors d'un affichage.

`@EqualsAndHashCode.Exclude` exclut ce champ de la comparaison `equals()` et du calcul `hashCode()`, pour éviter la même boucle infinie lors d'une comparaison entre deux objets.


```

package com.example.midwa.model;

import jakarta.persistence.*;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;
import com.fasterxml.jackson.annotation.JsonIgnore;
import java.time.LocalDateTime;

@Entity
@Table(name = "users")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String username;

    @Column(nullable = false)
    private String password;

    @Column(nullable = false)
    private int age;

    @Column(nullable = false)
    private String country;

    @Column(nullable = true)
    private LocalDateTime lastPixelPlacedAt;

    @Column(nullable = true)
    private LocalDateTime lastClickerSyncAt;

    @OneToOne(mappedBy = "user")
    @lombok.ToString.Exclude
    @lombok.EqualsAndHashCode.Exclude
    @JsonIgnore
    private Wallet wallet;

    @OneToOne(mappedBy = "user", cascade = CascadeType.ALL)
    @lombok.ToString.Exclude
    @lombok.EqualsAndHashCode.Exclude
    @JsonIgnore
    private UserStats userStats;
}

```

5 Déploiement

Afin de rendre ce projet disponible au plus grand nombre nous avons mis en place un déploiement sur un raspberry pie lié à un nom de domaine. Nous avons effectué les redirections depuis OVH afin de faire pointer notre sous-domaine dwa sur notre raspberry, nous avons ensuite installé Caddy, un reverse proxy afin de pouvoir exposer seulement les ports 80 et 443. Nous avons ensuite setup le certificat https, et finalement déployé les trois instances de notre projet (Serveur de front, Serveur de back, Base de donnée) sur la machine distante. Après plusieurs erreurs liés au CORS ORIGIN et au Websocket nous avons réussi à avoir une installation fonctionnelle.

6 Répartition du travail

6.1 Damien (Chef de projet + Lead dev)

- Implémentation structure principal du project
- Sécuriter
- API et Websocket
- Cliker (tout)
- Rplace (tout ce qui a pas était fait par les autres)
- Login / Register
- Statistique
- Debugueur et fix totaliter du projet
- Gestion Erreur serveur
- Fichier de configuration
- Tips
- UI/UX
- Scoreboard Frontend
- Correction du front pour l'affichage unifié

Utilistion de l'ia pour le css et les certaine fonctionaliter supplmenetaire

Asset provenant de @vectorsmarket15

6.2 Lukas (Dev + UI Designer)

- Système de modification de données personnelles
- Update visuelle des infos sur les pixels
- Classement back-end
- Système de stat front (avec Damien)
- Visuel pour les couleurs (Controle + visuel)
- Couleurs à débloquent
- Implémentation de la pipette
- Implémentation de la brush CARRE
- Implémentation de la brush ROND

6.3 Titouan (DevOps)

- Mise en place initial des docker et de la BDD
- Mise à jour du websocket en vue de prendre en charge le scoreboard
- Ajout du scoreboard Backend
- Ajout du système de gestion de l'âge des pixels globale
- Correction du front pour l'affichage unifié
- Fixes divers
- Mise en place du Reverse proxy pour le déploiement
- Déploiement du projet et gestion des problèmes de sécurités et chiffrement
- Écriture du rapport